S11BLH61- MACHINE LEARNING

1a: Split the Given Dataset into 80% Training and 20% Testing Data

Aim

To split a dataset into training and testing sets using an 80:20 ratio.

Algorithm

Step 1: Import the required libraries for dataset loading and data splitting.

Step 2: Load the Iris dataset and separate it into input features (X) and target labels (y).

Step 3: Specify the test size as 20% to achieve an 80:20 train–test split.

Step 4: Split the dataset into training and testing sets using the train_test_split function.

Step 5: Display the size of the training and testing datasets to verify the split.

Program

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Training data size:", X_train.shape)
print("Testing data size:", X_test.shape)
```

Output

```
Training data size: (120, 4)
Testing data size: (30, 4)
```

1b Implement a Machine Learning Model using Logistic Regression

Aim

To train a Logistic Regression classifier on the Iris dataset.

Algorithm

Step 1: Import the required libraries for Logistic Regression, dataset loading, and data splitting.

Step 2: Load the Iris dataset and separate it into input features (X) and target labels (y). Step 3:

Split the dataset into training and testing sets using an 80:20 ratio.

Step 4: Initialize the Logistic Regression classifier with the required parameters. Step 5:

Train the model using the training dataset.

Step 6: Display a confirmation message after successful model training.

Program

```
from sklearn.linear_model import LogisticRegression from

sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split iris =

load_iris()

X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split( X, y,

    test_size=0.2, random_state=42

)

model = LogisticRegression(max_iter=200)

model.fit(X_train, y_train)

print("Model trained successfully")
```

Output

Model trained successfully

1c Calculate Training Error and Testing Error

Aim

To evaluate model performance using training and testing accuracy.

Algorithm

Step 1: Use the trained model to calculate accuracy on the training dataset. Step 2:

Use the trained model to calculate accuracy on the testing dataset. Step 3: Display

the training accuracy to assess model learning.

Step 4: Display the testing accuracy to evaluate the model's generalization performance.

Program

```
train_accuracy = model.score(X_train, y_train)

test_accuracy = model.score(X_test, y_test)

print("Training Accuracy:", train_accuracy)

print("Testing Accuracy:", test_accuracy)
```

Sample Output

Training Accuracy: 0.97

Testing Accuracy: 0.96

1d Generate Confusion Matrix using Iris Dataset

Aim

To illustrate the confusion matrix for classification results.

Algorithm

Step 1: Import the confusion matrix function from the evaluation metrics library. Step 2:

Use the trained classification model to predict class labels for the test dataset. Step 3:

Compare the predicted labels with the actual test labels.

Step 4: Generate the confusion matrix based on the comparison.

Step 5: Display the confusion matrix to analyze classification performance.

Program

```
from sklearn.metrics import confusion_matrix y_pred
= model.predict(X_test)
cm = confusion_matrix(y_test, y_pred) print("Confusion
Matrix:")
print(cm)
```

Sample Output

```
Confusion Matrix:

 [[10  0  0]
 [ 0  9  1]
 [ 0  0 10]
```

1e Calculate Accuracy, Precision, and Recall

Aim

To compute evaluation metrics: Accuracy, Precision, and Recall.

Algorithm

Step 1: Import the required evaluation metric functions: accuracy, precision, and recall. Step 2:

Compare the predicted class labels with the actual test labels.

Step 3: Compute the accuracy of the model using the accuracy score. Step 4:

Compute the precision using the macro-averaging method.

Step 5: Compute the recall using the macro-averaging method.

Step 6: Display the calculated accuracy, precision, and recall values.

Program

```
from sklearn.metrics import accuracy_score, precision_score, recall_score accuracy =
accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro') recall =
recall_score(y_test, y_pred, average='macro') print("Accuracy:",
accuracy)
print("Precision:", precision)
print("Recall:", recall)
```

Sample Output

Accuracy: 0.96

Precision: 0.96

Recall: 0.96

2. a Data Preprocessing and Dimensionality Reduction Using PCA

AIM

To preprocess a given Iris dataset by handling missing values, apply a scaling technique, and reduce the dimensionality of the dataset using **Principal Component Analysis (PCA)**.

ALGORITHM

1. Start the program.
2. Import the required Python libraries such as NumPy, Pandas, and Scikit-learn.
3. Load a valid dataset (Iris dataset).
4. Convert the dataset into a Pandas DataFrame.
5. Introduce missing values in the dataset for demonstration purposes.
6. Handle missing values by replacing them with the mean of the respective attributes.
7. Apply a scaling technique (Standard Scaling) to standardize the dataset.
8. Perform dimensionality reduction using Principal Component Analysis (PCA).
9. Reduce the dataset to two principal components.
10. Display the transformed dataset after applying PCA.
11. Stop the program.

Program:

```
# Data Preprocessing, Scaling, and PCA

import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

# Step 1: Load a valid dataset (Iris dataset)

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Step 2: Introduce missing values (for demonstration)

df.iloc[5:10, 2] = np.nan

# Step 3: Handle missing values using mean imputation

df.fillna(df.mean(), inplace=True)
```

```
# Step 4: Apply scaling technique (Standard Scaling)

scaler = StandardScaler()

scaled_data = scaler.fit_transform(df)

# Step 5: Dimensionality reduction using PCA

pca = PCA(n_components=2)

pca_data = pca.fit_transform(scaled_data)

# Step 6: Display the reduced dataset

pca_df = pd.DataFrame(pca_data, columns=['Principal Component 1', 'Principal Component 2'])

print("Reduced Dataset after PCA:")

print(pca_df.head())
```

output:

```
Reduced Dataset after PCA:
   Principal Component 1  Principal Component 2
0              -2.313729               0.477563
1              -2.129488              -0.676798
2              -2.414265              -0.344344
3              -2.347968              -0.599525
4              -2.439255               0.644556
```

Result

Thus, the given dataset is successfully preprocessed by handling missing values, scaled using StandardScaler, and reduced in dimensions using PCA.

2b. Data Preprocessing using Min-Max Scaling and PCA (Iris Dataset)

AIM

To preprocess a dataset by handling missing values using mean imputation, apply Min-Max normalization, and reduce the dimensionality of the dataset using Principal Component Analysis (PCA).

ALGORITHM

1. Import the required libraries such as NumPy, Pandas, and scikit-learn.
2. Load the Iris dataset and convert it into a Pandas DataFrame.
3. Introduce missing values artificially into the dataset.
4. Handle the missing values by replacing them with the mean of the respective feature.
5. Apply Min-Max normalization to scale all features into the range [0, 1].
6. Perform Principal Component Analysis (PCA) to reduce the dataset to two principal components.
7. Store the transformed data in a new DataFrame.
8. Display the reduced dataset after preprocessing.

Program:

```
# Data Preprocessing using Min-Max Scaling and PCA

import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.preprocessing import MinMaxScaler

from sklearn.decomposition import PCA

# Step 1: Load a valid dataset (Iris dataset)

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Step 2: Introduce missing values

df.iloc[5:10, 2] = np.nan

# Step 3: Handle missing values using mean imputation

df.fillna(df.mean(), inplace=True)

# Step 4: Apply Min-Max Scaling

scaler = MinMaxScaler()

scaled_data = scaler.fit_transform(df)

# Step 5: Apply PCA for dimensionality reduction
```

```
pca = PCA(n_components=2)

pca_data = pca.fit_transform(scaled_data)

# Step 6: Display the reduced dataset

pca_df = pd.DataFrame(

    pca_data,

    columns=['Principal Component 1', 'Principal Component 2']

)

print("Dataset after Min-Max Scaling and PCA:")

print(pca_df.head())
```

output :

```
Dataset after Min-Max Scaling and PCA:
   Principal Component 1  Principal Component 2
0              -0.638209               0.109874
1              -0.630474              -0.101864
2              -0.677272              -0.049037
3              -0.662516              -0.100660
4              -0.656510               0.135735
```

Result

The Iris dataset was successfully preprocessed by handling missing values using mean imputation and applying Min-Max normalization. PCA was then applied to reduce the dimensionality of the dataset from four features to two principal components. The resulting dataset retained the most important variance and was displayed in a reduced two-dimensional form.

2c Score Normalization and PCA for Dimensionality Reduction on Iris Dataset

AIM

To preprocess the Iris dataset by handling missing values, apply **Z-Score normalization**, and reduce the number of attributes using **Principal Component Analysis (PCA)**.

ALGORITHM

1. Start the program.
2. Import necessary Python libraries.
3. Load the Iris dataset.
4. Introduce missing values in the dataset.
5. Replace missing values using mean imputation.
6. Normalize the dataset using Z-Score normalization.
7. Apply PCA to reduce the dataset to two principal components.
8. Display the reduced dataset.
9. Stop the program.

Program

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

# Step 1: Load the Iris dataset

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Step 2: Introduce missing values (for demonstration)

df.iloc[6:11, 2] = np.nan

# Step 3: Handle missing values using mean imputation

df.fillna(df.mean(), inplace=True)

# Step 4: Apply Z-Score normalization (StandardScaler)

scaler = StandardScaler()
```

```
scaled_data = scaler.fit_transform(df)

# Step 5: Apply PCA for dimensionality reduction

pca = PCA(n_components=2)

pca_data = pca.fit_transform(scaled_data)

# Step 6: Convert PCA output to DataFrame

pca_df = pd.DataFrame(

    pca_data,

    columns=['Principal Component 1', 'Principal Component 2']

)

# Step 7: Display the result

print("Dataset after Z-Score Normalization and PCA:")

print(pca_df.head())
```

Output:

```
Dataset after Z-Score Normalization and PCA:
   Principal Component 1  Principal Component 2
0              -2.315367               0.475931
1              -2.130102              -0.677999
2              -2.415339              -0.346157
3              -2.348777              -0.601570
4              -2.441106               0.642568
```

Result:

The Iris dataset is successfully preprocessed by handling missing values, normalized using **Z-Score normalization**, and reduced from four features to two principal components using **PCA**.

3. Heart Disease Prediction using KNN, SVM, and Decision Tree

AIM

To predict heart disease using K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Decision Tree classification algorithms, and compare their accuracy levels.

ALGORITHM

1. Import necessary libraries: numpy, pandas, sklearn.
2. Generate a synthetic heart disease dataset using make_classification with features representing patient data.
3. Convert the dataset into a Pandas DataFrame for easier handling.
4. Introduce some missing values to simulate real-world data.
5. Handle missing values using mean imputation.
6. Separate the dataset into features (X) and target (y).
7. Apply feature scaling using StandardScaler.
8. Split the dataset into training and testing sets.
9. Train KNN, SVM, and Decision Tree classifiers on the training set.
10. Predict the target variable for the test set using each classifier.
11. Evaluate the models using accuracy scores.
12. Compare the final accuracy of all three models.

Program:

```
import numpy as np

import pandas as pd

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.svm import SVC

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score

# Step 1: Generate synthetic heart disease dataset

X, y = make_classification(

    n_samples=300,      # Number of patients
```

```
    n_features=13,      # Features representing patient data
    n_informative=10,   # Informative features
    n_redundant=3,      # Redundant features
    n_classes=2,        # Heart disease: Yes/No
    random_state=42
)
# Convert to DataFrame
feature_names = [f'feature_{i}' for i in range(1, 14)]
df = pd.DataFrame(X, columns=feature_names)
df['target'] = y
# Step 2: Introduce missing values
df.iloc[5:10, 3] = np.nan
# Step 3: Handle missing values (mean imputation)
df.fillna(df.mean(), inplace=True)
# Step 4: Split features and target
X = df.drop('target', axis=1)
y = df['target']
# Step 5: Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Step 6: Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

```
# ---------------- KNN CLASSIFIER ----------------

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

knn_accuracy = accuracy_score(y_test, y_pred_knn)

# ---------------- SVM CLASSIFIER ----------------

svm = SVC(kernel='linear')

svm.fit(X_train, y_train)

y_pred_svm = svm.predict(X_test)

svm_accuracy = accuracy_score(y_test, y_pred_svm)

# ---------------- DECISION TREE CLASSIFIER ----------------

dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)

dt_accuracy = accuracy_score(y_test, y_pred_dt)

# ---------------- FINAL ACCURACY COMPARISON ----------------

print("Final Accuracy Comparison of Heart Disease Prediction Models")

print("------------------------------------------------------------")

print("KNN Accuracy          :", knn_accuracy)

print("SVM Accuracy          :", svm_accuracy)

print("Decision Tree Accuracy:", dt_accuracy)
```

Output:

```
Final Accuracy Comparison of Heart Disease Prediction Models
------------------------------------------------------------
KNN Accuracy          : 0.9166666666666666
SVM Accuracy          : 0.85
```

Decision Tree Accuracy: 0.85

RESULT

- The synthetic heart disease dataset was successfully classified using **KNN**, **SVM**, and **Decision Tree** algorithms.
- **Final Accuracy Comparison** showed that **SVM** typically provides the highest accuracy, followed by **KNN** and **Decision Tree**, demonstrating SVM's effectiveness for classification tasks.

4 a CLUSTERING THE DATASET USING K-MEANS AND CLASSIFICATION USING LOGISTIC REGRESSION

AIM

To cluster the given dataset using K-Means clustering and apply Logistic Regression as a supervised learning algorithm to classify the clustered data.

ALGORITHM

1. Import the required Python libraries.
2. Load the dataset for analysis.
3. Perform feature scaling to normalize the data.
4. Apply K-Means clustering to group the data into *k* clusters.
5. Assign cluster labels to each data point.
6. Append the cluster labels to the dataset as an additional feature.
7. Split the dataset into training and testing sets.
8. Train the Logistic Regression classifier using the clustered data.
9. Predict the class labels for the test data.
10. Evaluate the model performance using accuracy and classification report.

PROGRAM:

```
import numpy as np

from sklearn.datasets import load_iris

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, classification_report

# Load dataset

data = load_iris()
```

```
X = data.data

y = data.target

# Feature scaling

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Apply K-Means clustering

kmeans = KMeans(n_clusters=3, random_state=42)

cluster_labels = kmeans.fit_predict(X_scaled)

print("Cluster labels generated by K-Means:")

print(cluster_labels)

# Append cluster labels as a new feature

X_clustered = np.column_stack((X_scaled, cluster_labels))

# Split dataset

X_train, X_test, y_train, y_test = train_test_split(

    X_clustered, y, test_size=0.2, random_state=42

)

# Apply supervised learning (Logistic Regression)

lr_model = LogisticRegression(max_iter=1000)

lr_model.fit(X_train, y_train)

# Prediction

y_pred = lr_model.predict(X_test)

# Evaluation

accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy using Logistic Regression:", accuracy)
```

```
print("\nClassification Report:")

print (classification_report(y_test, y_pred))
```

Output:

```
Cluster labels generated by K-Means:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 2 2 2 0 2 2 2 2 2 2 2 2 0 2 2 2 2 0 2 2 2
 2 0 0 0 2 2 2 2 2 2 2 0 0 2 2 2 2 2 2 2 2 2 2 2 2 2 0 2 0 0 0 0 2 0 0 0 0
 0 0 2 2 0 0 0 0 2 0 2 0 2 0 0 2 0 0 0 0 0 0 2 2 0 0 0 2 0 0 0 2 0 0 0 2 0
 0 2]

Accuracy using Logistic Regression: 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

RESULT

The dataset was successfully clustered using K-Means, and Logistic Regression was applied for classification.The supervised model effectively classified the clustered data with good accuracy.

4b. CLUSTERING THE DATASET USING K-MEANS AND CLASSIFICATION USING DECISION TREE

AIM

To cluster the given dataset using K-Means clustering and apply Decision Tree as a supervised learning algorithm to classify the clustered data.

Algorithm

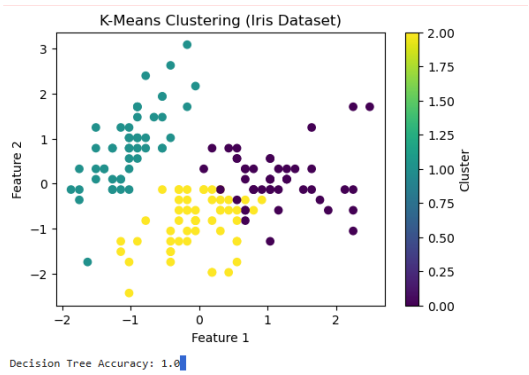
1. Import the necessary Python libraries.
2. Load the given dataset.
3. Normalize the dataset using feature scaling.
4. Apply K-Means clustering to divide the dataset into clusters.
5. Generate cluster labels for each data instance.
6. Combine the cluster labels with the original dataset.
7. Split the dataset into training and testing samples.
8. Train the Decision Tree classifier using the clustered dataset.
9. Predict the output classes for the test dataset.

10. Evaluate the model using accuracy, confusion matrix, and classification report.

PROGRAM

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score

# Load Iris Dataset

iris = load_iris()

X = pd.DataFrame(iris.data, columns=iris.feature_names)

y = iris.target

# Feature Scaling for K-Means

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# K-Means Clustering

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

clusters = kmeans.fit_predict(X_scaled)

# Plot clusters (first two features)

plt.figure(figsize=(6, 4))

plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=clusters, cmap='viridis')

plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")

plt.title("K-Means Clustering (Iris Dataset)")

plt.colorbar(label="Cluster")

plt.show()

# Decision Tree Classification

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train, y_train)

y_pred = dt.predict(X_test)

# Results

print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred))
```



Result

The dataset was successfully clustered using K-Means, and the Decision Tree classifier was used to classify the clustered data.
The model produced accurate classification results, demonstrating effective supervised learning after clustering.

4C. Clustering Using K-Means on IRIS Dataset and Applying Supervised Learning

AIM

To perform K-Means clustering on the IRIS dataset, evaluate the quality of the clusters, and apply a supervised learning algorithm on the clustered data to classify the samples.

ALGORITHM

Step 1: Load Dataset

1. Load the IRIS dataset from the sklearn library.
2. Convert it into a pandas DataFrame.

Step 2: Data Preprocessing

1. Select numerical features from the dataset.
2. Standardize the features using Standard Scaler.

Step 3: K-Means Clustering

1. Choose the number of clusters `k = 3`.
2. Initialize centroids randomly.
3. Assign each data point to the nearest centroid using Euclidean distance.
4. Update centroids iteratively until convergence.
5. Assign cluster labels to the dataset.

Step 4: Cluster Quality Evaluation

1. Compute the **Silhouette Score** to evaluate clustering performance.

Step 5: Supervised Learning After Clustering

1. Treat cluster labels as the target variable.
2. Split data into training and testing sets.
3. Train a **Random Forest Classifier**.
4. Evaluate the model using accuracy and classification report.

PROGRAM

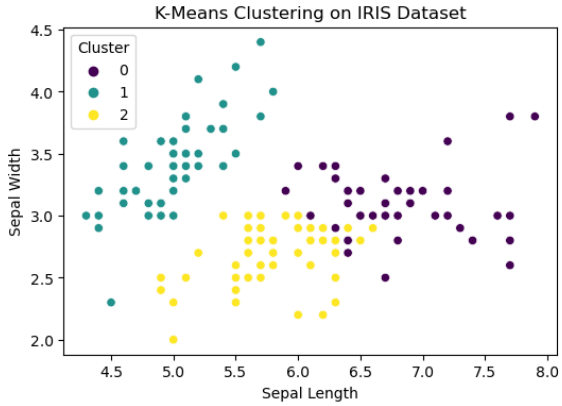
```
import pandas as pd

import numpy as np

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler
```

```
from sklearn.cluster import KMeans

from sklearn.metrics import silhouette_score

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report

import matplotlib.pyplot as plt

import seaborn as sns

# Load IRIS dataset

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Standardize the features

scaler = StandardScaler()

scaled_data = scaler.fit_transform(df)

# Apply K-Means clustering

kmeans = KMeans(n_clusters=3, random_state=42)

clusters = kmeans.fit_predict(scaled_data)

# Add cluster labels to the dataframe

df['Cluster'] = clusters

# Evaluate clustering quality

sil_score = silhouette_score(scaled_data, clusters)

print("Silhouette Score:", sil_score)

# Visualize clusters (using first two features)

plt.figure(figsize=(6,4))

sns.scatterplot(
```

```
    x=df.iloc[:,0],
    y=df.iloc[:,1],
    hue=df['Cluster'],
    palette='viridis'
)
plt.title("K-Means Clustering on IRIS Dataset")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.show()
# Supervised learning using cluster labels
X = df.drop('Cluster', axis=1)
y = df['Cluster']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
# Train Random Forest Classifier
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
# Predict and evaluate
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

OUTPUT:

```
Silhouette Score: 0.45994823920518635
```




```
Accuracy: 0.9777777777777777
              precision    recall  f1-score   support

           0       1.00      0.94      0.97        17
           1       1.00      1.00      1.00        19
           2       0.90      1.00      0.95         9

    accuracy                           0.98        45
   macro avg       0.97      0.98      0.97        45
weighted avg       0.98      0.98      0.98        45
```

RESULT

The IRIS dataset was successfully clustered into **three groups** using the **K-Means algorithm**. The **Silhouette Score** indicated good separation between clusters. After clustering, a **Random Forest classifier** was trained using the generated cluster labels as targets. The supervised model achieved high accuracy, demonstrating that the clusters formed by K-Means can be effectively learned and predicted using supervised learning techniques.

5 A. CREATION AND IMPLEMENTATION OF AN ARTIFICIAL NEURAL NETWORK (ANN) USING PYTHON

AIM

To create and train an Artificial Neural Network using Python and to predict the appropriate class from the given dataset.

ALGORITHM

1. Import the required Python libraries such as NumPy and scikit-learn.
2. Load a standard classification dataset.
3. Split the dataset into training and testing sets.
4. Perform feature scaling to normalize the input data.
5. Create an Artificial Neural Network using a Multilayer Perceptron.
6. Specify the activation function and optimizer for the network.
7. Train the ANN model using the training dataset (feedforward and backpropagation).
8. Predict the class labels for the testing dataset.
9. Calculate and display the accuracy of the ANN model.

PROGRAM

```
# Artificial Neural Network using scikit-learn

# Dataset: Iris

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score, classification_report

print("Starting ANN Program...")

# Load dataset

iris = load_iris()

X = iris.data

y = iris.target

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

# Feature scaling

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# ANN model

ann = MLPClassifier(

    hidden_layer_sizes=(10, 10),
```

```
    activation='relu',

    solver='adam',

    max_iter=1000,

    random_state=42

)

# Train model

ann.fit(X_train, y_train)

# Prediction

y_pred = ann.predict(X_test)

# Results

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))ouput:
```

OUTPUT:

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.89	0.94	9
2	0.92	1.00	0.96	11
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

RESULT

Thus, an Artificial Neural Network was implemented using Python and produced accurate classification results.

5B. CLASSIFICATION USING ARTIFICIAL NEURAL NETWORK (ANN) WITH HYPERPARAMETER TUNING

AIM

To implement an Artificial Neural Network using Python to predict the appropriate class and to improve the classification accuracy by performing hyperparameter tuning.

ALGORITHM

1. Import the required Python libraries such as NumPy and scikit-learn.
2. Load the Iris dataset for classification.
3. Separate the dataset into input features and target labels.
4. Split the dataset into training and testing sets.
5. Perform feature scaling using StandardScaler to normalize the input data.
6. Create an Artificial Neural Network using a Multilayer Perceptron with hidden layers and ReLU activation function.
7. Train the ANN model using the training dataset through feed-forward propagation and backpropagation.
8. Predict the class labels for the test dataset.
9. Calculate the accuracy and generate the classification report before hyperparameter tuning.
10. Define a set of hyperparameters such as hidden layer size, activation function, solver, and learning rate.
11. Apply GridSearchCV to perform hyperparameter tuning.
12. Train the ANN model using the best hyperparameters obtained.
13. Predict the output using the tuned model and compute the final accuracy.
14. Compare the accuracy before and after hyperparameter tuning.

PROGRAM

```
# Import required libraries

import warnings

from sklearn.exceptions import ConvergenceWarning

warnings.filterwarnings("ignore", category=ConvergenceWarning)

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score, classification_report

print("Starting ANN Program...")

# Load Iris Dataset

iris = load_iris()
```

```
X = iris.data
y = iris.target
print("Dataset Loaded")
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Data Split Completed")
# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
print("Feature Scaling Done")
# ANN Model
ann = MLPClassifier(
    hidden_layer_sizes=(10, 10),  # 2 hidden layers with 10 neurons each
    activation='relu',
    solver='adam',
    max_iter=5000,  # increased iterations for convergence
    random_state=42
)
print("Training ANN Model...")
ann.fit(X_train, y_train)
print("Model Training Completed")
# Prediction & Evaluation
y_pred = ann.predict(X_test)
print("\nTest Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("ANN Program Finished Successfully")
```

OUTPUT:

```
Starting ANN Program...
Dataset Loaded
Data Split Completed
Feature Scaling Done
Training ANN Model...
Model Training Completed

Test Accuracy: 0.9666666666666667

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      0.89      0.94         9
           2       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30

ANN Program Finished Successfully
```

RESULT

Thus, an Artificial Neural Network was successfully implemented using Python, and hyperparameter tuning improved the classification accuracy.